

Raft Consensus Server Setup & Implementation Guide

1. Core Server States

Each server maintains the following state variables:

Persistent State (Must be saved to disk before responding to RPCs)

- **current_term**: Latest term the server has seen (initialized to 0, increases monotonically).
- **voted_for**: Candidate ID that received a vote in the current term (or **null** if none).
- **log[]**: Log entries; each entry contains a command for the state machine and the term when the entry was received by the leader (first index is 1).

Volatile State (Lost on crash)

- **commit_index**: Index of the highest log entry known to be committed (initialized to 0).
- **last_applied**: Index of the highest log entry applied to the state machine (initialized to 0).

Volatile State on Leaders (Reinitialized after every election)

- **next_index[]**: For each server, the index of the next log entry to send to that server (initialized to leader's last log index + 1).
- **match_index[]**: For each server, the index of the highest log entry known to be replicated on the server (initialized to 0).

2. Server Roles and State Machine Rules

Servers can be in one of three states: **Follower**, **Candidate**, or **Leader**.

All Servers

- If **commit_index** > **last_applied**: increment **last_applied** and apply **log[last_applied]** to the state machine.
- If an RPC request or response contains a term $T > \text{current_term}$: set **current_term** = T and immediately convert to a Follower.

Followers

- Respond to RPCs from candidates and leaders.
- If the election timeout elapses without receiving an **AppendEntries** RPC from the current leader or granting a vote to a candidate: convert to Candidate.

Candidates

- On conversion to a candidate, start an election:
 1. Increment **current_term**.
 2. Vote for self.
 3. Reset the election timer.
 4. Send **RequestVote** RPCs to all other servers.

- If votes are received from a majority of servers: become Leader.
- If an **AppendEntries** RPC is received from a new leader: convert to Follower.
- If the election timeout elapses: start a new election.

Leaders

- Upon election: send initial empty **AppendEntries** RPCs (heartbeats) to each server; repeat during idle periods to prevent follower election timeouts.
- If a command is received from a client: append the entry to the local log, and respond after the entry is applied to the state machine.
- If the last log index $\geq$ **next_index** for a follower: send an **AppendEntries** RPC with log entries starting at **next_index**.
 - If successful: update **next_index** and **match_index** for that follower.
 - If **AppendEntries** fails because of a log inconsistency: decrement **next_index** and retry.
- If there exists an N such that $N > \text{commit_index}$, a majority of **match_index[i]** $\geq N$, and **log[N].term == current_term**: set **commit_index** = N .

3. Remote Procedure Calls (RPCs)

A. RequestVote RPC

- **Arguments:**
 1. **term**: Candidate's term.
 2. **candidate_id**: Candidate requesting vote.
 3. **last_log_index**: Index of candidate's last log entry.
 4. **last_log_term**: Term of candidate's last log entry.
- **Results:**
 1. **term**: Current term, for candidate to update itself.
 2. **vote_granted**: True means the candidate received the vote.
- **Receiver Implementation Rules:**
 1. Reply false if **term** < **current_term**.
 2. If **voted_for** is null or **candidate_id**, and the candidate's log is at least as up-to-date as the receiver's log, grant the vote.

B. AppendEntries RPC

- **Arguments:**
 1. **term**: Leader's term.
 2. **leader_id**: So the follower can redirect clients.
 3. **prev_log_index**: Index of the log entry immediately preceding the new ones.
 4. **prev_log_term**: Term of the **prev_log_index** entry.
 5. **entries[]**: Log entries to store (empty for heartbeat).
 6. **leader_commit**: Leader's **commit_index**.
- **Results:**
 1. **term**: Current term, for leader to update itself.

2. **success:** True if follower contained an entry matching `prev_log_index` and `prev_log_term`.
- **Receiver Implementation Rules:**
 1. Reply false if `term < current_term`.
 2. Reply false if the log doesn't contain an entry at `prev_log_index` whose term matches `prev_log_term`.
 3. If an existing entry conflicts with a new one (same index but different terms), delete the existing entry and all that follow it.
 4. Append any new entries not already in the log.
 5. If `leader_commit > commit_index`, set `commit_index = min(leader_commit, index of last new entry)`.